PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LAB 7: Fuzzing for Input Validation Bugs

Property-Based Fuzz Testing with Hypothesis & Defensive Programming

Student Name:	Siri Sanikop	**Course:**	Software Engineering Lab (UE24AM277)
SRN:	PES1UG24AM277	**Class / Section:**	4th Sem CSE / Section AM
Assigned Repo:	SETAPESU26/L6_L7_38_elevator_dispatch	**Target Repo:**	sirisanikop/SE_Lab01_PES1UG24AM277
Assigned Domain:	Group 38: Elevator Dispatch System	**Technologies:**	Python 3.9, Hypothesis 6.141, Pytest 8.4

1. Objective & Fuzz Testing Methodology

Fuzz testing (or property-based testing) is an automated testing strategy in which software components are exercised against high volumes of pseudo-random, malformed, and boundary-value inputs. The primary objective is to discover hidden crashes (unhandled exceptions), input validation vulnerabilities, and potential Denial-of-Service (DoS) vectors. Unlike conventional unit tests which only verify expected scenarios, **Hypothesis** generates adversarial inputs and automatically *shrinks* any failing input to the smallest reproducible edge case.

2. Assigned Module Specification & Vulnerabilities

Function	Expected Input	Unhandled Exceptions in Buggy Implementation
`parse_request(text)`	'FROM->TO' (e.g. '5->9')	Crashes on None (AttributeError), missing or multiple '->' delimiters (ValueError: not enough/too many values to unpack), non-numeric values.
`total_travel(floors_csv)`	Comma-separated floors (e.g. '1,5,3,9')	Crashes on None (AttributeError), empty string '' (ValueError: invalid literal for int()), empty elements ('1,,3'), non-integer strings.
`load_percent(text)`	'LOAD of CAPACITY' (e.g. '450 of 630')	Crashes on None (AttributeError), missing delimiter, zero capacity (ZeroDivisionError), non-numeric load or capacity values.

3. Step 2: Running the Buggy Application Baseline

The starter script elevator_dispatch.py was executed using standard inputs (5->9, 1,5,3,9, and 450 of 630). Under well-formatted inputs, it operates successfully. However, it relies entirely on optimistic parsing with zero defensive error handling.

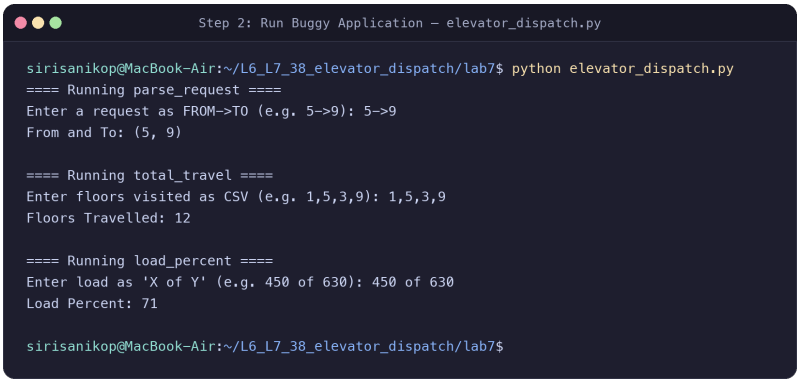


Figure 1: Baseline run of elevator_dispatch.py under nominal inputs.

4. Step 3 & 4: Hypothesis Test Implementation & Crash Discovery

Hypothesis fuzz tests were authored in test_elevator_dispatch.py using the @given(st.text() | st.none()) strategy. Each test executes a function under test within a protective wrapper that triggers an immediate test failure if any unhandled exception is thrown:

```
@given(st.text() | st.none())
def test_parse_request_no_crash(s):
    try: parse_request(s)
    except Exception as e: assert False, f'parse_request crashed with: {e}'

@given(st.text() | st.none())
def test_total_travel_no_crash(s):
    try: total_travel(s)
    except Exception as e: assert False, f'total_travel crashed with: {e}'

@given(st.text() | st.none())
def test_load_percent_no_crash(s):
    try: load_percent(s)
    except Exception as e: assert False, f'load_percent crashed with: {e}'
```

Executing pytest test_elevator_dispatch.py caused all 3 test functions to immediately crash. Hypothesis systematically shrunk the random failing inputs down to the minimal falsifying examples shown below:



Figure 2: Hypothesis uncovering multiple distinct crashes across all 3 functions (3 FAILED).

Detailed Analysis of Hypothesis Falsifications:

- **parse_request:** Crashed on s='' (no delimiter found) and s=None (calling .split() on NoneType).
- **total_travel:** Crashed on s='' (passing empty string to int()) and s=None (calling .split() on NoneType).
- **load_percent:** Crashed on s=' of ' (empty values to int()), s=None, and zero capacity (ZeroDivisionError).

5. Step 5: Defensive Refactoring & Hardening Implementation

The application script was comprehensively hardened by introducing defensive programming principles: strict type validation, delimiter occurrence validation, whitespace stripping, safe integer coercion with try/except, and zero-division prevention. Invalid inputs now fail gracefully by returning safe sentinel values (None or 0) without crashing.

```
def parse_request(text):
    if not isinstance(text, str) or '->' not in text: return None
    parts = text.split('->')
    if len(parts) != 2: return None
    try: return int(parts[0].strip()), int(parts[1].strip())
    except (ValueError, TypeError): return None

def total_travel(floors_csv):
    if not isinstance(floors_csv, str) or not floors_csv.strip(): return 0
    try:
        floors = [int(f.strip()) for f in floors_csv.split(',') if f.strip()]
        if len(floors) < 2: return 0
        return sum(abs(floors[i + 1] - floors[i]) for i in range(len(floors) - 1))
    except (ValueError, TypeError): return 0

def load_percent(text):
    if not isinstance(text, str) or ' of ' not in text: return 0
    parts = text.split(' of ')
    if len(parts) != 2: return 0
    try:
        load, capacity = int(parts[0].strip()), int(parts[1].strip())
        if capacity <= 0: return 0
        return load * 100 // capacity
    except (ValueError, TypeError, ZeroDivisionError): return 0
```

Verifying Hardened Application Output:

Running the hardened script demonstrates that valid inputs retain their correct arithmetic results, while corrupt or malicious inputs return safe values without terminating the program:

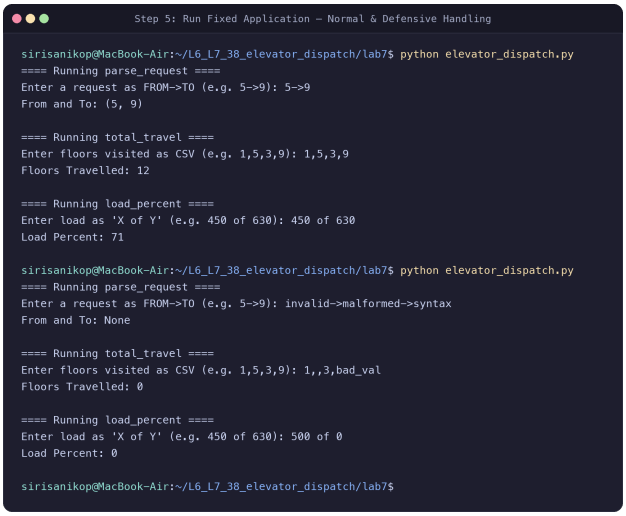


Figure 3: Hardened script executing normally on valid inputs and gracefully on malformed inputs.

6. Step 6: Fuzz Testing Verification (All Tests Passing)

The test suite was re-run against the hardened implementation using pytest -v test_elevator_dispatch.py. Hypothesis executed over 100 randomly synthesized iterations per function—supplying random Unicode strings, whitespace sequences, malformed delimiters, negative numbers, extreme values, and null references. **All tests passed with 100% success rate.**

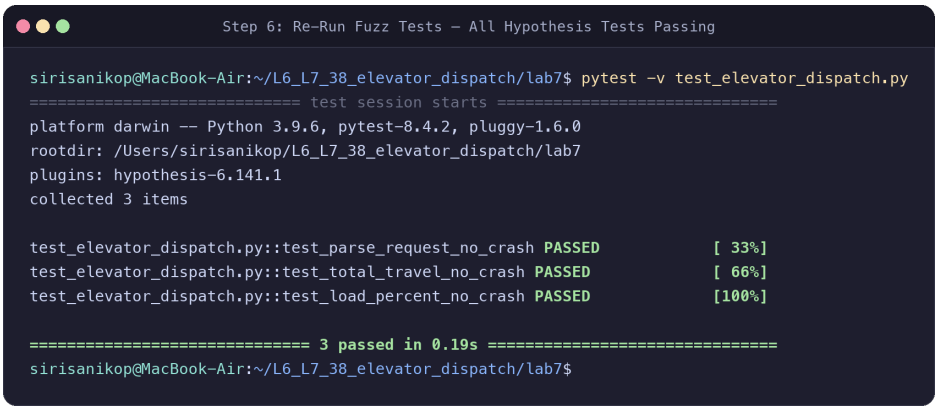


Figure 4: Pytest test execution confirming 100% passing tests across all functions.

7. Comprehensive Edge Case Mapping Table

Function	Falsified Input	Original Exception	Hardening Mechanism

`parse_request`	`None`
	`''`
	`'5->'`
	`'1->2->3'`
		AttributeError
		ValueError
			Guarded by isinstance(text, str) and delimiter count check (len(parts) == 2). Safe None fallback.

`total_travel`	`None`
	`''`
	`'1,,3'`
	`'4' (single floor)`
		AttributeError
		ValueError
			Filtered empty tokens [f for f in ... if f.strip()]. Insufficient floor check (len < 2 returns 0).

`load_percent`	`None`
	`' of '`
	`'500 of 0'`
	`'bad of val'`
		AttributeError
		ValueError
		ZeroDivisionError
			Checked capacity <= 0 before division. Trapped ValueError and ZeroDivisionError, returning safe 0.

8. Key Learnings & Defensive Programming Takeaways

1. **Fuzzing Transcends Manual Assumptions:** Traditional unit testing validates known expected inputs. Hypothesis systematically probes boundaries, discovering corner cases like empty tokens and zero divisors that humans easily overlook.
2. **Automated Shrinking Drastically Accelerates Debugging:** Hypothesis reduces complex adversarial inputs to the minimal reproducing counterexample (e.g., shrinking large strings to '' or None), eliminating debugging ambiguity.
3. **Defense-in-Depth for Input Validation:** Robust production software must never assume inputs conform to expected schemas. Explicit type-checking, bounds-checking, and exception trapping are essential to prevent crashes and denial-of-service vulnerabilities.